

Soma de Vetores em CUDA

Comparação de desempenho entre CPU sequencial e GPU paralela

Wésio Filho

Arquitetura de Computadores — 5º Período
Prof. Newarney T. da Costa

2026

$$C[i] = A[i] + B[i]$$

Objetivo

Implementar a soma de vetores em duas versões e comparar desempenho:

- **CPU:** versão sequencial em C, usando laço `for`.
- **GPU:** versão paralela em CUDA, usando uma thread por elemento.
- **Métrica principal:** tempo médio e speedup.

Ideia-chave: cada elemento pode ser calculado de forma independente.

CPU vs GPU: Ideia Geral

CPU

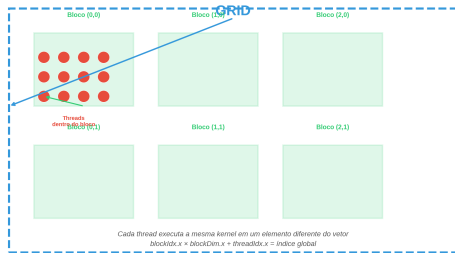
- Poucos núcleos mais complexos.
- Excelente para tarefas sequenciais.
- No código deste trabalho, percorre o vetor elemento por elemento.

GPU

- Muitos núcleos simples.
- Excelente para tarefas repetitivas e independentes.
- Executa milhares de threads em paralelo.

Analogia rápida: CPU é um guerreiro forte; GPU é um exército de clones fazendo a mesma missão em partes diferentes.

Modelo CUDA: Grid, Blocos e Threads



- **Thread:** unidade mínima; calcula um elemento do vetor.
- **Bloco:** grupo de threads executado em um SM da GPU.
- **Grid:** conjunto de blocos lançados para uma kernel.

Implementação CPU

```
static void somar_vetores_cpu(const float *A,  
                             const float *B,  
                             float *C,  
                             int N) {  
    for (int i = 0; i < N; i++) {  
        C[i] = A[i] + B[i];  
    }  
}
```

- Percorre os elementos em ordem.
- Cada iteração depende apenas de $A[i]$ e $B[i]$.
- Tempo medido com `clock_gettime(CLOCK_MONOTONIC)`.

Implementação GPU em CUDA

```
__global__ void somaVetores(const float *A,
                           const float *B,
                           float *C,
                           int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < N) {
        C[i] = A[i] + B[i];
    }
}
```

- `__global__`: função executada na GPU.
- Cada thread calcula seu índice global `i`.
- `if (i < N)` evita acesso fora do vetor.

Metodologia

- Ambiente principal: notebook Acer Nitro V15 com CachyOS.
- CPU: 13th Gen Intel Core i5-13420H.
- GPU: NVIDIA GeForce RTX 4050 Laptop GPU, 6 GB VRAM.
- Compilação CPU: `gcc -std=c11 -O2 -Wall -Wextra`.
- Compilação GPU: `nvcc -O2`.

Tamanhos testados

1.000; 100.000; 1.000.000; 10.000.000 elementos — 5 execuções por tamanho.

CPU

- Mede o laço sequencial.
- Usa `clock_gettime`.
- Resultado validado elemento a elemento.

GPU

- Mede transferência + kernel + retorno.
- Usa `cudaEvent`.
- Resultado validado após copiar para o host.

$$\text{speedup} = \frac{\text{tempo CPU}}{\text{tempo GPU}}$$

Resultados Numéricos

N	CPU (ms)	GPU (ms)	Speedup
1.000	0,0042	0,1681	0,02x
100.000	0,3207	0,2076	1,54x
1.000.000	3,1630	0,3349	9,45x
10.000.000	31,4850	0,5917	53,21x

- Para vetor pequeno, CPU vence por causa do overhead da GPU.
- Para vetores grandes, o paralelismo massivo domina.

Gráfico: Tempo CPU vs GPU

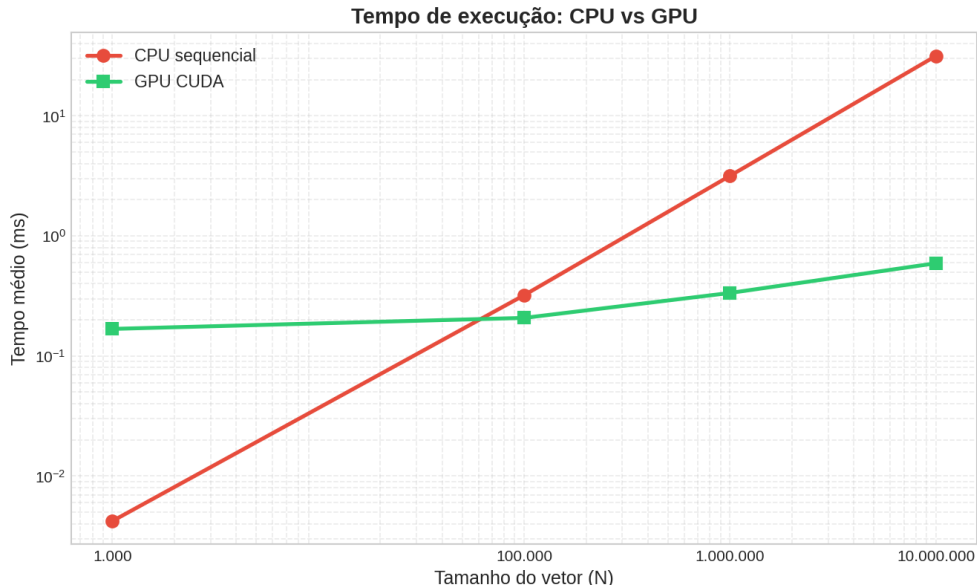
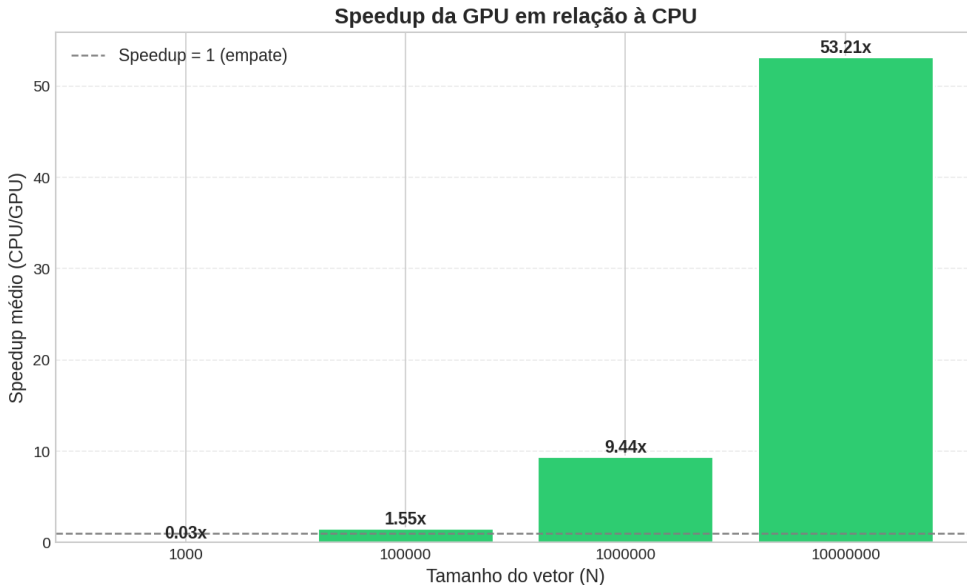


Gráfico: Speedup



Conclusão

- A soma de vetores é altamente paralelizável porque cada posição é independente.
- A GPU perde em entradas pequenas por causa do overhead inicial.
- A GPU vence em entradas grandes, chegando a **53,21x** no maior teste.
- O trabalho consolidou conceitos de **kernel**, **threads**, **blocos**, **grid**, **memória host/device** e **speedup**.

Entrega

Repositório GitHub + relatório PDF + slides PDF + resultados e declaração de uso de IA.